IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

Project – High Level Design

on

Natural Language Interface for Manufacturing Infrastructure

Student Name: Anaina Vadakkanchery

Industry Mentor: Mr.Shashank **Guided by: Mr.Praveen**

Table of Contents

1. Introduction……3
   1.1. Scope of the document……3
   1.2. Intended Audience……...3
   1.3. System overview……..4

2. System Design……4
   2.1. Application Design……..5
   2.2. Process Flow……6
   2.3. Information Flow……7
   2.4. Components Design……8
   2.5. Key Design Considerations……..8
   2.6. API Catalogue……..8

3. Data Design……9
   3.1. Data Model……..9
   3.2. Data Access Mechanism……..9
   3.3. Data Retention Policies……..10
   3.4. Data Migration……...10

4. Interfaces……...10

5. State and Session Management……..11

6. Caching……..11

7. Non-Functional Requirements……...11
   7.1. Security Aspects……..11
   7.2. Performance Aspects……...12

8. References……12

1. Introduction

InfraChat is a web based natural language interface for manufacturing infrastructure that allows users to query real manufacturing data using plain English commands. Users can type questions like "show failed machines" or search a specific machine by its Product ID and receive instant answers without any technical knowledge.

The system is built using Python Flask as the backend, HTML CSS and JavaScript as the frontend, RapidFuzz for fuzzy string matching and Pandas for dataset processing. The system features a professional homepage with a floating chatbot and a dedicated dashboard page displaying live manufacturing statistics and charts.

This document provides the high level architectural overview of the InfraChat system including its structure, components, data flow and design decisions.

1.1 Scope of the Document

This document covers the high level design of the InfraChat web application including the frontend presentation layer, the Python Flask backend and the manufacturing dataset integration. It covers the system context, use case overview, three tier architecture, application design, process flow, information flow, components design, API catalogue, data design, interfaces, state management, caching and non functional requirements.

This document does not cover the detailed function level implementation which is described in the Low Level Design document. It does not cover production deployment or user authentication.

1.2 Intended Audience

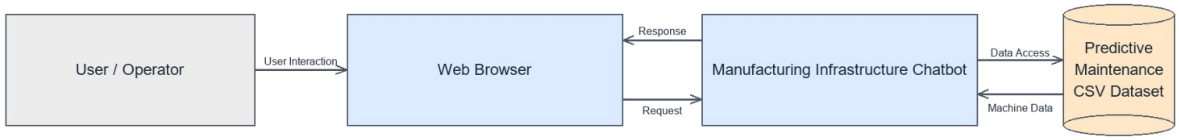
- Internal project guide and academic mentor
- Project evaluation panel members
- Developers who want to understand the overall system architecture
- Students studying natural language processing and web development

1.3 System Overview

InfraChat consists of two main interfaces. The homepage provides a professional landing page with a sidebar navigation, hero section, statistics bar, about section, machines overview cards and footer. A floating robot button opens the chatbot assistant from any page. The dashboard page provides live KPI cards, gauge charts, a status chart, a failure mode chart and a machine data table.

The core intelligence is a natural language chatbot that uses fuzzy string matching to understand user queries even with spelling mistakes. A risk scoring engine evaluates each machine as Safe, Elevated, High Warning or Critical based on sensor readings.

System Context Diagram



The User interacts with the system through a Web Browser. The browser sends HTTP requests to the Manufacturing Infrastructure Chatbot system which accesses the Predictive Maintenance CSV Dataset to retrieve machine data and return answers to the user.

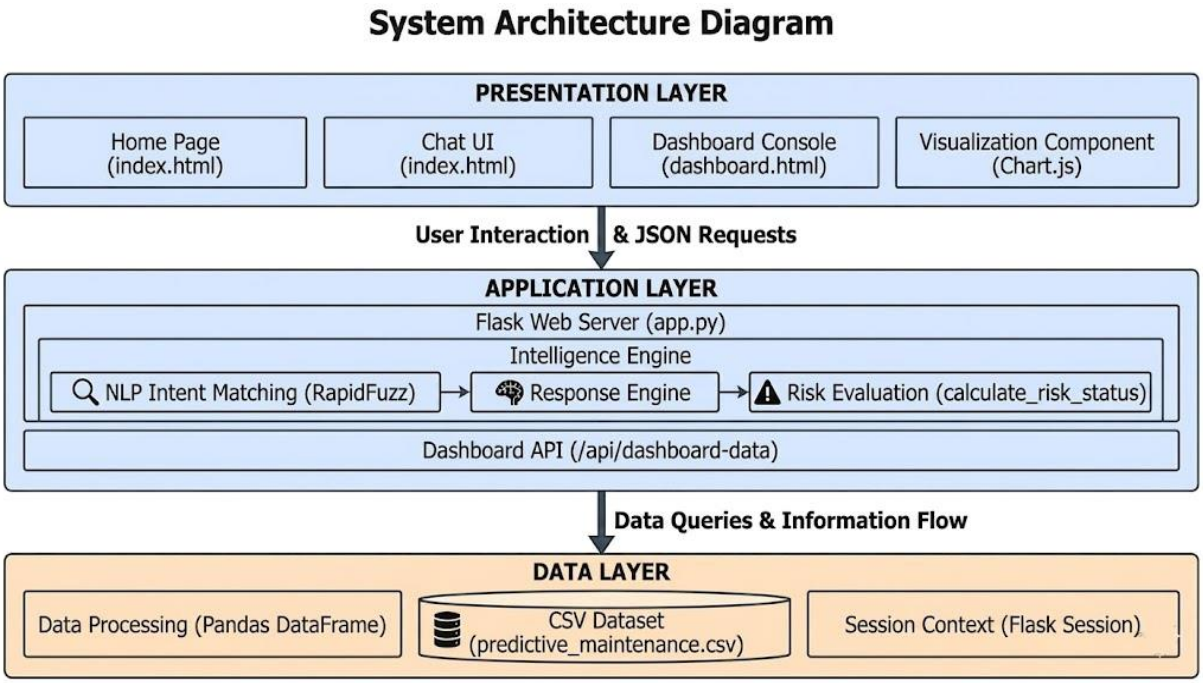
2. System Design

InfraChat follows a Three Tier Application Architecture.

Presentation Layer:
Handles user interaction. Includes the home page (index.html), dashboard (dashboard.html), chatbot interface, and Chart.js for displaying data visually.

Application Layer:
Processes all logic. Built using Flask (app.py), it includes intent matching with RapidFuzz, response generation using get_response(), risk calculation using calculate_risk_status(), and provides data through the /api/dashboard-data API.

Data Layer:
Manages data storage and access. Uses the CSV file (predictive_maintenance.csv), which is loaded into a Pandas DataFrame in memory, along with session context for handling follow-up queries.



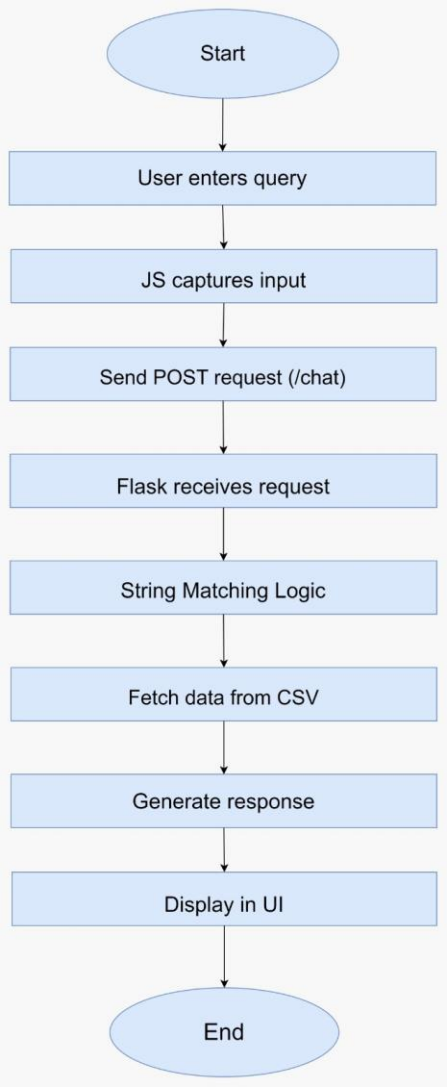
2.1 Application Design

The Flask application in app.py initializes the server, loads the dataset using pd.read_csv() and exposes three routes. The matches() function performs two-layer string matching using direct substring check first and then RapidFuzz fuzzy matching with an 80% similarity threshold. The get_response() function checks for a machine Product ID pattern using regex and if found returns full machine details. If no ID is found it uses matches() to identify the query intent and filters the

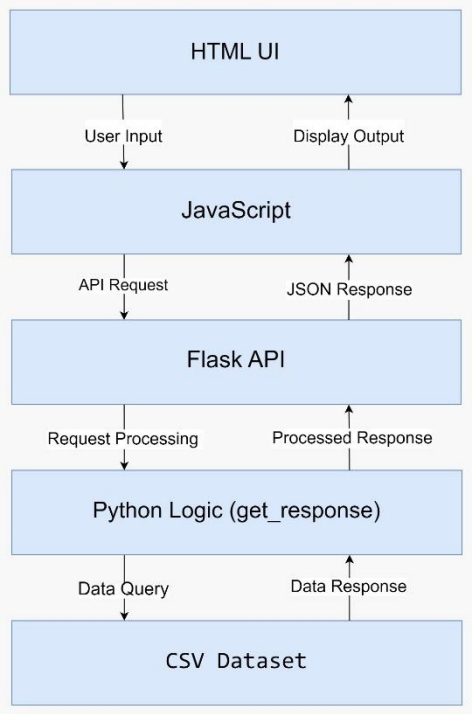
Pandas DataFrame accordingly. The calculate_risk_status() function evaluates machine health by checking tool wear and torque values against dataset thresholds.

2.2 Process Flow

The user opens the website, sees the homepage, clicks the robot button to open the chatbot, types a query and submits it. JavaScript sends a POST request to /chat. Flask processes the query through get_response(), filters the dataset and returns a JSON response. JavaScript displays the answer as a message bubble in the chat window.



2.3 Information Flow



- User Input: The user types a query into the HTML UI.
- API Request: JavaScript captures the input and sends an API Request to the backend.
- Request Processing: The Flask API receives the request and sends it for Request Processing.
- Data Query: The Python Logic (get_response) executes a Data Query against the CSV Dataset.
- Data Response: The CSV Dataset returns the raw information as a Data Response.
- Processed Response: The Python logic formats the data and sends a Processed Response back to the API.
- JSON Response: The Flask API sends a JSON Response to the frontend.
- Display Output: JavaScript handles the data and provides the Display Output on the screen for the user.

2.4 Components Design

Frontend:
Includes chat interface (index.html) and dashboard (dashboard.html). Uses script.js for chatbot logic and dashboard.js with Chart.js for visualization.

Backend:
Built with Flask to handle requests. Uses get_response() for replies, find_best_command() for intent matching, calculate_risk_status() for health analysis, and /api/dashboard-data for dashboard data.

Data:
Uses a Pandas DataFrame (df) with ~10,000 records loaded from predictive_maintenance.csv.

2.5 Key Design Considerations

- Speed: Loads data into Pandas once at startup for instant responses.
- Flexibility: Uses RapidFuzz so the chatbot understands typos.
- Separation: Frontend and Backend are independent and talk via JSON.
- Clarity: Risk levels are based on clear rules (Heuristics), not "black-box" AI.

2.6 API Catalog

Endpoint	Method	Purpose
/	GET	Loads the Home Page.
/chat	POST	Processes user questions and returns answers.
/api/dashboard-data	GET	Sends all stats and chart data to the dashboard.
/dashboard	GET	Loads the Analytics Dashboard

3. Data Design

3.1 Data Model

The dataset is predictive_maintenance.csv containing 10,000 machine records with 10 columns.

Data Model Table

Column Name	Data Type	Description
UDI	Integer	Unique row identifier for each dataset record
Product ID	String	Unique machine or product identifier
Type	String	Machine type: L, M, or H
Air temperature [K]	Float	Measured air temperature in Kelvin
Process temperature [K]	Float	Measured process temperature in Kelvin
Rotational speed [rpm]	Integer	Machine rotational speed
Torque [Nm]	Float	Torque produced by the machine
Tool wear [min]	Integer	Tool wear time in minutes
Target	Integer	0 = working, 1 = failed
Failure Type	String	Failure category such as No Failure or Power Failure

3.2 Data Access Mechanism

- The dataset is loaded once at application startup using pd.read_csv() and stored in a global Pandas DataFrame (df).
- All queries are performed using Pandas boolean indexing (e.g., df[df['Target'] == 1]).

- The system operates in a read-only manner, ensuring the original dataset remains unchanged.

3.3 Data Retention Policies

- No personal or user data is collected or stored.
- Chat history is maintained only in the browser and is cleared on page refresh.
- Flask session stores only the last searched Product ID for follow-up queries.
- The dataset is not modified during execution.

3.4 Data Migration

- No migration mechanism is required in the current system.
- Dataset updates can be handled by replacing the CSV file with the same structure and restarting the application.

4. Interfaces

- User Interface:
  - Homepage includes chatbot, sections like overview and statistics.
  - Dashboard displays KPI cards, charts, and a data table.
- Frontend to Backend:
  - Uses fetch() API to send POST requests to /chat with JSON input.
  - Receives responses in JSON format.
  - Dashboard data is retrieved via GET request to /api/dashboard-data.

- Backend to Data:
  - Flask interacts with the Pandas DataFrame using indexing operations.
  - No external database is used.

5.State and Session Management

- Flask session stores the last searched Product ID (session['last_machine']).
- No additional server-side state is maintained.
- Chat history is not stored on the server.

6. Caching

- The Pandas DataFrame acts as an in-memory cache.
- CSV file is read only once at startup.
- This reduces disk access and improves performance.
- No external caching mechanisms are used.

7. Non-Functional Requirements

7.1 Security Aspects

- Application runs on localhost and is not publicly exposed.
- User inputs are sanitized (lowercase and trimmed).
- No personal data is collected or stored.
- No authentication is implemented (academic use).

7.2 Performance Aspects

- Data is loaded into memory, avoiding repeated file access.
- Pandas queries execute in a few milliseconds.
- Fuzzy matching is highly efficient.
- Overall system response time is typically under 500 ms.

8. References

1. Manufacturing Dataset: *Machine Predictive Maintenance Classification*, Kaggle.
2. Flask Documentation: https://flask.palletsprojects.com/
3. Pandas Documentation: https://pandas.pydata.org/
4. RapidFuzz Documentation: https://rapidfuzz.github.io/
5. Chart.js Documentation: https://www.chartjs.org/
6. Python 3.x Documentation: https://docs.python.org/